

Predictor de demanda para redes eléctricas

Metodología CRISP-DM

Fase: Comprensión del Negocio

La comprensión del negocio en este proyecto, consisten en entender que la optimización de la red eléctrica nacional depende de la capacidad de anticipar la demanda para integrar eficientemente las energías renovables y reducir costes operativos.

El objetivo principal es transformar datos brutos de múltiples fuentes en información que permita identificar patrones de consumo, definiendo como éxito técnico la implementación de una arquitectura híbrida funcional y como éxito de negocio la creación de una capa Oro con la que crear un dashboard interactivo que correlacione las variables para la toma de decisiones.

Como éxito financiero del proyecto, nos basaremos en mantener un coste operativo lo más cercano a cero, ya que minimizaremos costes con la implementación híbrida para el procesamiento pesado de los datos

Fase: Comprensión de los datos

En esta fase se han identificado y explorado las 4 fuentes de datos que utilizaremos para sacar la información, asegurando que la información sea suficiente y de calidad para los objetivos del negocio.

Los datos se han extraído de forma automatizada mediante AWS lambda y almacenado inicialmente en un Data Lake en formato bruto (Capa bronce):

De la Api de la Demanda eléctrica hemos recogido 4879 registros diarios con granularidad diaria en formato json.

De la Api de precios, hemos obtenido 110804 registros de precios horario (€/MWh) en formato json

De la Aemet, hemos obtenido un histórico desde 2013 hasta la actualidad descargados en archivos comprimidos que contienen .xls

De la Api de los festivos, hemos obtenido datos de festivos nacionales y locales en formato json (para identificar días no laborables)

Fase: Comprensión de los datos

En esta fase, se realizó un análisis de los datos obtenidos de las diversas fuentes con el objetivo de conocer la estructura de los datos e identificar posibles problemas de calidad y ver si existen relaciones entre las distintas variables

Para la API de la red eléctrica, recogimos los datos de un json y observamos que los datos eran:

datetime: fecha y hora del registro.

value: valor de la demanda eléctrica.

percentage: porcentaje asociado al valor.

type: tipo de indicador energético

Para la API de los precios, vimos que los datos eran:

Type: Tipo de indicador energético

Value: Valor de demanda eléctrica

Percentage: Porcentaje asociado

Datetime: Fecha y hora del registro

Para la API de los festivos nos encontramos con:

Value: Precio de la electricidad

Percentage: Porcentaje asociado al valor

Datetime: Fecha y hora del registro

Para la web de la Aemet, el formato de los datos es el siguiente:

Fecha: Día del registro meteorológico

indicativo: Código de la estación meteorológica

nombre: Nombre de la estación

provincia: Provincia de la estación

altitud: Altitud de la estación

tmed: Temperatura media

tmin: Temperatura mínima

tmax: Temperatura máxima

prec: Precipitación

velmedia: Velocidad media del viento

dir: Dirección del viento

sol: Horas de sol

Fase: Preparación de los datos

En esta fase, obtenemos los datos que previamente hemos recogido de sus respectivas fuentes y almacenamos en S3 y lo bajamos para trabajar con los datos en un sistema distribuido en local, de manera que reducimos costes.

Contaremos con Aws para almacenar los datos en bruto en la capa bronce y en local, las capas plata y oro.

Para la capa Bronce, se almacenarán los datos mediante lambdas, y con una cola de mensajes, estaremos informados en caso de que la ingesta de datos no haya sido correcta.

☐	recogidaDatosAPIRedElectrica	-	Zip	Python 3.14
☐	recogidaDatosFestivos	-	Zip	Python 3.14
☐	recogidaDatosAPIPrecios	-	Zip	Python 3.14
☐	recogidaDatosHtml	-	Zip	Python 3.14

 **Simple Queue Service**
Colas de mensajes administradas por SQS

Detalles información

Nombre
 errores_proyecto

Cifrado
Clave de Amazon SQS (SSE-SQS)

Más

Tipo
Estándar

URL
 https://sqs.us-east-1.amazonaws.com/891377024918/errores_proyecto

ARN
 arn:aws:sqs:us-east-1:891377024918:errores_proyecto

Cola de mensajes fallidos
-



Fuente : Demanda

```
ruta_demanda = "/workspace/silver/demanda/"
ruta_parquet = "/workspace/silver/demanda/DEMANDA_FINAL"
bucket_name = "angel-barrientos-simo-prada-big-data"
prefix_s3 = "bronze/demanda/"

os.makedirs(ruta_demanda, exist_ok=True)
for archivo in os.listdir(ruta_demanda):
    if archivo.endswith(".json"):
        os.remove(
            os.path.join(
                ruta_demanda,
                archivo
            )
        )

s3 = boto3.client("s3")
respuesta = s3.list_objects_v2(
    Bucket=bucket_name,
    Prefix=prefix_s3
)

print("Descargando archivos demanda")
for obj in respuesta.get("Contents", []):
    key = obj["Key"]
    if key.endswith(".json"):
        nombre_archivo = key.split("/")[-1]
        s3.download_file(bucket_name, key, os.path.join(ruta_demanda, nombre_archivo))
```

Creamos la Carpeta que contendrá los archivos descargados

Descargamos los archivos que se encuentran en la capa bronce de S3

Procesamos los archivos con Spark, transformando estructuras anidadas en tablas

```
spark = (
    SparkSession.builder
        .appName("silverDemanda")
        .master("local[*]")
        .config("spark.driver.memory", "2g")
        .getOrCreate()
)

try:
    print("Procesando archivos demanda")
    archivos_json = [
        file for file in os.listdir(ruta_demanda)
        if file.endswith(".json")
    ]

    dfs_procesados = []
    for nombre_archivo in archivos_json:
        ruta_completa = os.path.join(ruta_demanda, nombre_archivo)
        print(f"Procesando: {nombre_archivo}")
        df_temp = spark.read.json(ruta_completa)
        df_limpio = (df_temp.select(explode(col("included")).alias("datos"))
                     .select(col("datos.attributes.title").alias("titulo"), explode(col("datos.attributes.values")).alias("v"))
                     .select(col("titulo"), col("v.value").alias("megavatio"), col("v.datetime").alias("fecha_hora"), lit(nombre_archivo).alias("archivo_origen"))
                     )
        dfs_procesados.append(df_limpio)

    if len(dfs_procesados) > 0:
        df_final = dfs_procesados[0]
        for i in range(1, len(dfs_procesados)):
            df_final = df_final.union(dfs_procesados[i])
        df_demanda_plata = (df_final.filter(col("titulo") == "Demanda")
                           .withColumn("fecha_hora_limpia", to_timestamp(substring(col("fecha_hora"), 1, 19)))
                           .withColumn("fecha_cruce", to_date(col("fecha_hora_limpia")))
                           .withColumn("megavatio", col("megavatio").cast("float"))
                           .withColumn("hora", split(col("fecha_hora_limpia").cast("string"), " ")[1])
                           .select("fecha_cruce", col("megavatio").alias("megavatio_dia"))
                           )
        print("Muestra demanda plata")
        df_demanda_plata.show(10, truncate=False)
        print("Schema")
        df_demanda_plata.printSchema()
        total = df_demanda_plata.count()
        print(f"Total registros: {total}")
```

Una vez sacamos la información útil de los datos, unificamos la información procesada, filtramos y limpiamos los datos.

Finalmente generamos el DataFrame final para el análisis.

Una vez tenemos el DataFrame, convertimos el contenido de este en un archivo .parquet, ya que es un formato muy ligero, lo que facilitara procesos posteriores

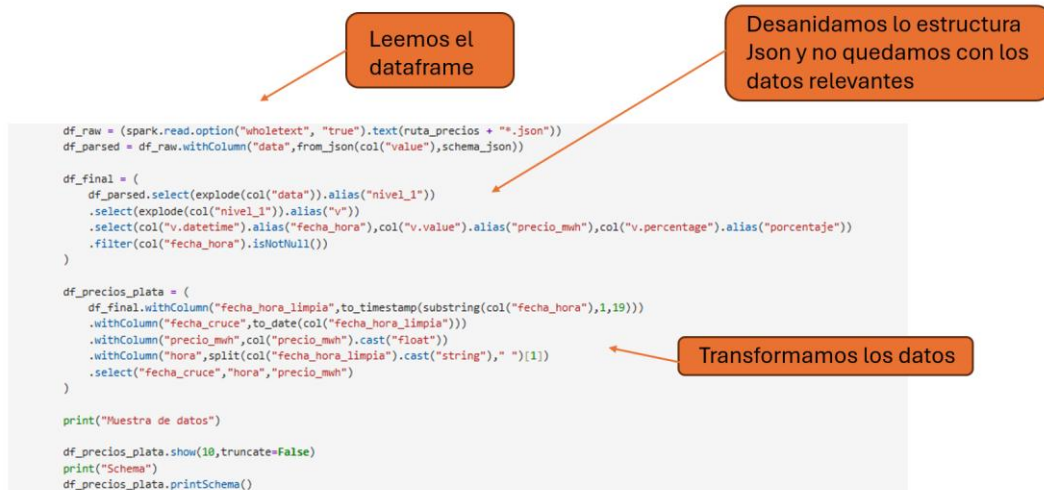
```
print("Guardando parquet demanda")
(
    df_demanda_plata.coalesce(1).write.mode("overwrite").parquet(ruta_parquet)
)
print(f"Parquet guardado en: {ruta_parquet}")
print(os.listdir(ruta_parquet))

else:
    print("No hay datos válidos")
else:
    print("No se encontraron JSON")

print("Borrando JSON")
for archivo in os.listdir(ruta_demanda):
    if archivo.endswith(".json"):
        os.remove(os.path.join(ruta_demanda, archivo))
print("PROCESO demandaPlata FINALIZADO")
```

Fuente : Precios

Una vez cogemos los datos de la capa bronce situada en S3:



Para la transformación de los datos, limpiamos la fecha y nos aseguramos de que el tipo de los datos sea el correcto

Al igual que con la Demanda, creamos un archivo .parquet y borramos los archivos descargados de S3.

Fuente: Festivos



Al igual que con las demás fuentes, recogemos los datos de S3 y recogemos los datos relevantes de los json , para las transformaciones iniciales, aparte de quedarnos solo con los datos que no sean nulos, convertimos la columna fecha a tipo date y creamos una columna es_festivo en función de si es vacaciones nacionales o no.(Lo convertimos a booleano tipo 0/1)

Fuente: Climas

Una vez descargamos los .rar de la capa bronce de aws, los descomprimos

```
print("Descomprimiendo archivos RAR")
archivos_rar = [f for f in os.listdir(ruta_base) if f.endswith(".rar")]

for rar_nombre in archivos_rar:
    ruta_rar = os.path.join(ruta_base, rar_nombre)
    try:
        with rarfile.RarFile(ruta_rar) as rf:
            rf.extractall(ruta_xls)

    except Exception as e:
        print(f"Error descomprimiendo {rar_nombre}: {e}")
```

Seleccionamos los archivos .rar que encontramos en el directorio y descomprimos

En caso de error informamos de ello

Comprobamos de la existencia de los xls

```
def procesar_xls(datos_binarios):
    ruta, contenido = datos_binarios
    try:
        pdf = pd.read_excel(io.BytesIO(contenido), engine="xlrd", header=None)
        columnas = ["Estacion", "Provincia", "TMax", "TMin", "TMed", "Racha", "VMax", "P00_24", "P00_06", "P06_12", "P12_18", "P18_24"]
        df_datos = pdf.iloc[5:].copy()
        df_datos = df_datos.iloc[:, :len(columnas)]
        df_datos.columns = columnas
        df_datos["Fecha"] = str(pdf.iloc[2, 0])
        return (df_datos.dropna(subset=["Estacion"]).astype(str).to_dict("records"))
    except Exception:
        return []

    try:
        print("Procesando archivos XLS")

        archivos_xls = [os.path.join(r, f)
                        for r, d, fs in os.walk(ruta_xls)
                        for f in fs
                        if f.lower().endswith(".xls")]

        if not archivos_xls:
            print("No hay archivos XLS")
            return

        rutas_spark = ", ".join([f"file://{os.path.abspath(f)}" for f in archivos_xls])
        df_clima_raw = (
            spark.sparkContext
            .binaryFiles(rutas_spark)
            .flatMap(procesar_xls)
            .toDF()
        )
```

1º Leemos cada archivo
2º Generamos un dataframe para cada archivo, indicando que las 5 primeras filas no tienen contenido
3º Indicamos el nombre de las columnas
4º Convertimos la Fecha al mismo formato que el resto de fuentes



```
df_verificacion = (df_clima_raw
    .withColumn("Fecha_Temp", split(col("Fecha"), "-")[1])
    .withColumn("Fecha_Temp", trim(col("Fecha_Temp")))
    .withColumn("Dia_Semana", split(col("Fecha_Temp"), "-")[0])
    .withColumn("Resto_Fecha", trim(split(col("Fecha_Temp"), "-")[1]))
    .withColumn("Dia_Mes", split(col("Resto_Fecha"), "-")[0])
    .withColumn("Mes_Nomina", lower(split(col("Resto_Fecha"), "-")[1]))
    .withColumn("Anio", split(col("Resto_Fecha"), "-")[2])
    .withColumn("TMax_Hora_aux", col("TMax"))
    .withColumn("TMax", split(col("TMax"), "-")[0])
    .withColumn("TMax_Hora", split(col("TMax_Hora_aux"), "-")[1])
    .withColumn("TMax_Hora", regexp_replace(col("TMax_Hora"), "-", ""))
    .withColumn("TMax_Hora", regexp_replace(col("TMax_Hora"), "\\", ""))
    .withColumn("TMin_Hora_aux", col("TMin"))
    .withColumn("TMin", split(col("TMin"), "-")[0])
    .withColumn("TMin_Hora", split(col("TMin_Hora_aux"), "-")[1])
    .withColumn("TMin_Hora", regexp_replace(col("TMin_Hora"), "-", ""))
    .withColumn("TMin_Hora", regexp_replace(col("TMin_Hora"), "\\", ""))
    .withColumn("VMax_aux", col("VMax"))
    .withColumn("VMax", split(col("VMax"), "-")[0])
    .withColumn("VMax_Hora", split(col("VMax_Hora_aux"), "-")[1])
    .withColumn("VMax_Hora", regexp_replace(col("VMax_Hora"), "-", ""))
    .withColumn("VMax_Hora", regexp_replace(col("VMax_Hora"), "\\", ""))
    .withColumn("Racha_aux", col("Racha"))
    .withColumn("Racha", split(col("Racha"), "-")[0])
    .withColumn("Racha_Hora", split(col("Racha_Hora_aux"), "-")[1])
    .withColumn("Racha_Hora", regexp_replace(col("Racha_Hora"), "-", ""))
    .withColumn("Racha_Hora", regexp_replace(col("Racha_Hora"), "\\", "")))

meses_dict = {
    "enero": "01",
    "febrero": "02",
    "marzo": "03",
    "abril": "04",
    "mayo": "05",
    "junio": "06",
    "julio": "07",
    "agosto": "08",
    "septiembre": "09",
    "octubre": "10",
    "noviembre": "11",
    "diciembre": "12"
}

mapping_expr = create_map(
    [
        lit(x)
        for x in [val for pair in meses_dict.items() for val in pair]
    ]
)
```

Realizamos las transformaciones

Mapeamos los meses

```

        df_clima_plata.coalesce(1).write.mode("overwrite").parquet(ruta_parquet)
    )
    print(f"Parquet guardado en: {ruta_parquet}")
    print(os.listdir(ruta_parquet))
else:
    print("No hay datos válidos")

print("Borrando archivos temporales")

for archivo in os.listdir(ruta_base):
    if archivo.endswith(".rar"):
        try:
            os.remove(os.path.join(ruta_base, archivo))
        except Exception as e:
            print(f"Error borrando RAR {archivo}: {e}")

```

Creamos el archivo
.parquet

Borramos .rar

Además de eliminar los archivos comprimidos, también eliminamos los xls que sacamos de la descompresión

Vemos que los archivos .parquet se han generado correctamente

/ ... / clima / CLIMA_FINAL /		/ ... / demanda / DEMANDA_FINAL /	
Name	Modified	Name	Modified
_SUCCESS	11m ago	_SUCCESS	25m ago
part-00000-b4efc0...	11m ago	part-00000-c9f3d...	25m ago

/ ... / festivos / FESTIVOS_FINAL /		/ ... / precios / PRECIOS_FINAL /	
Name	Modified	Name	Modified
_SUCCESS	24m ago	_SUCCESS	25m ago
part-00000-bf309...	24m ago	part-00000-f0203a...	25m ago

Una vez tenemos todos los datos, nos preparamos para unir los archivos .parquet en uno solo

Leemos cada archivo

```
listaRutas = ["precios/PRECIOS_FINAL", "demanda/DEMANDA_FINAL", "festivos/FESTIVOS_FINAL", "clima/CLIMA_FINAL"]
contador = 0
for rutas in listaRutas:
    for archivo in os.listdir(f"/workspace/silver/{rutas}"):
        if archivo.lower().endswith(".parquet"):
            rutaCompleta = f"/workspace/silver/{rutas}/{archivo}"
            if contador == 0:
                rutaPartPrecio = rutaCompleta
            elif contador == 1:
                rutaPartDemanda = rutaCompleta
            elif contador == 2:
                rutaPartFestivos = rutaCompleta
            elif contador == 3:
                rutaPartClima = rutaCompleta
            contador += 1

spark = (
    SparkSession.builder
    .appName("goldEnergia")
    .master("local[*]")
    .config("spark.driver.memory", "4g")
    .getOrCreate()
)

df_precios = spark.read.parquet(rutaPartPrecio)
df_demanda = spark.read.parquet(rutaPartDemanda)
df_festivos = spark.read.parquet(rutaPartFestivos)
df_clima = spark.read.parquet(rutaPartClima)
print("\nDATOS LEÍDOS")
```

Realizamos las últimas transformaciones

```
columnas_clima = ["TMed", "TMax", "TMin", "Racha", "P00_24"]
for c in columnas_clima:
    df_clima = (
        df_clima.withColumn(c, when(isnan(col(c)), None).otherwise(col(c)))
    )

df_clima_agg = (
    df_clima.groupBy("fecha_cruce")
    .agg(
        avg("TMed").alias("temp_media"),
        avg("TMax").alias("temp_max_media"),
        avg("TMin").alias("temp_min_media"),
        avg("Racha").alias("racha_media"),
        avg("P00_24").alias("precipitacion_media")
    )
)

df_precios_agg = (
    df_precios
    .groupBy("fecha_cruce")
    .agg(
        avg("precio_mwh").alias("precio_medio_dia")
    )
)

df_festivos = (
    df_festivos.dropDuplicates(["fecha_cruce"])
)

df_gold = (df_demanda.join(df_festivos, ["fecha_cruce"], "left"))

df_gold = (df_gold.join(df_clima_agg, ["fecha_cruce"], "left"))

df_gold = (df_gold.join(df_precios_agg, ["fecha_cruce"], "left"))
```

Unimos los Dataframes por la columna común que es Fecha_cruce

Seleccionamos las columnas finales que vamos a usar

```
ruta_gold = "/workspace/gold/ENERGIA_GOLD"
df_gold = df_gold.select(col("fecha_cruce"), col("es_festivo"), col("temp_media"), col("temp_max_media"), col("temp_min_media"), col("racha_media"), col("precipitacion_media"), col("precio_medio_dia"), col("anio"), col("mes"), col("dia"))
```

```
df_gold.coalesce(1).write.mode("overwrite").parquet(ruta_gold)
```

Guardamos el resultado

/ gold / ENERGIA_GOLD /	
Name	Modified
_SUCCESS	23m ago
part-00000-f4ef...	23m ago

Una vez tenemos los datos en el mismo archivo, realizamos un muestreo de los datos, comprobando que el resultado es el esperado

[fecha_crucce]	[es_festivo]	[temp_media]	[temp_max_media]	[temp_min_media]	[fecha_media]	[precipitacion_media]	[precio_medio_dia]	[anio]	[mes]	[fin_de_semana]	[rango_temperatura]	[precio_alto]	[VAR_003]	[TIVO_megavatio_dia]
2014-01-01	1	9.31871347565372	12.661988381583896	5.972368426708101	40.38659793814433	3.7743783675821973	5.008758028411548	2014	1	0	6.689619874875797	10	585187.7	
2014-01-02	0	11.1163949846658099	13.724026802280821	8.764956939584922	39.08588489315908	5.1821785704471133	25.28883352247874	2014	1	0	4.98753664827308	10	715794.1	
2014-01-03	0	12.184671535979222	14.4635363684380879	9.397372263974518	37.84957264957265	3.080883648602706	39.924166679382324	2014	1	0	5.566131484455561	10	712191.1	
2014-01-04	0	9.405384612610707	13.49984612281596	5.389846152411857	70.1669658886894	8.484837575925553	4.992883350817363	2014	1	1	8.189999784040803	10	679475.5	
2014-01-05	0	8.5194791388839	12.584934842338655	4.452708632502976	45.49148351616438	1.25044511723788	4.091666692574345	2014	1	1	8.132218849795568	10	641269.4	
2014-01-06	1	9.38649719244416	13.8751461852712231	4.88444445358674	22.410034682076124	0.8855858594357444	56.202916866589444	2014	1	0	8.01026393715481	10	684080.25	
2014-01-07	0	11.86674267592741	15.422481167114169	6.786802932847338	28.694974803466204	1.2766816122916782	47.88541638851166	2014	1	0	8.71639823586683	10	753530.75	
2014-01-08	0	11.263478258381719	13.78979682225767	6.152318841715654	30.912521448823326	0.9718330816895729	41.28758002866294	2014	1	0	10.218268848510112	10	788952.3	
2014-01-09	0	10.161876852217141	15.185804480424368	5.127272733889592	25.359861591695902	1.2245588850711717	49.822883332414124	2014	1	0	10.86562168842775	10	783659.25	
2014-01-10	0	9.38649719244416	13.8751461852712231	4.88444445358674	22.410034682076124	0.8855858594357444	56.202916866589444	2014	1	0	8.588798174179255	10	752076.2	
2014-01-11	0	9.28837159648332	13.465880872612824	5.0932179018102515	19.93688088259385	0.16218611616723821	58.277883273965904	2014	1	1	8.3725829788802572	10	707795.5	
2014-01-12	0	9.333529418263922	13.68378787164432	4.957962697748646	30.28672268907563	1.3611764731652596	37.66541688174642	2014	1	1	8.725824959415785	10	659961.8	
2014-01-13	0	8.00845193680817	12.1884548675714219	5.083443322359575	37.22806889699656	3.288056133033694	38.82166638856435	2014	1	0	6.1895215452118435	10	785574.7	
2014-01-14	0	8.5392593851725	11.895157618031117	5.265329520284788	43.16368601801669	3.228193821273361	33.35288314657211	2014	1	0	6.539828809746329	10	881855.7	
2014-01-15	0	9.408143903622524	12.397697832944582	6.39971228742864	36.872268897563826	2.0421597537689387	49.856249888738	2014	1	0	5.997985604201718	10	798151.06	
2014-01-16	0	8.525808824829511	11.481741348822592	5.6383628687043186	48.486486486486484	4.879944199806862	41.77958325544993	2014	1	0	5.763562866177973	10	797720.8	
2014-01-17	0	7.4318142874748644	10.7328571833218	4.876285711931331	43.484848484848484	3.395614841189788	48.78978322558444	2014	1	0	6.61888886390469	10	792553.9	
2014-01-18	0	7.52578228059113	10.26642496565598	4.77881356835381	48.267768595841325	7.014183387875471	43.674166679382324	2014	1	1	5.487429484828679	10	717442.5	
2014-01-19	0	6.487591513789854	9.38579545118211	3.583664788736647	44.18833333333333	6.384126998247688	33.87083339691162	2014	1	1	5.882138678845463	10	678883.0	
2014-01-20	0	6.563015656940544	9.86273114491323	3.261593165889532	41.96666666666667	2.492612481896889	37.813329472815	2014	1	0	6.081137979987298	10	799958.5	
2014-01-21	0	7.879519781620945	10.786864428125461	3.3687344725837698	34.33057851239669	3.318695649472268	50.78833325368474	2014	1	0	7.426129547621691	10	823668.6	
2014-01-22	0	8.007571436860326	11.034571421061242	4.978800809134412	38.09531772575251	6.068585875948872	46.998749574025474	2014	1	0	6.95657141192683	10	821130.6	
2014-01-23	0	7.79875213844952	11.668838021582987	3.9264872684532552	39.47	1.637808962802114	44.464583273965904	2014	1	0	7.743342761129732	10	889113.56	
2014-01-24	0	8.79225354861131	13.15450784471986	4.42168014371153	44.81333333333333	2.868488587040773	48.44833895946694	2014	1	0	8.712816989890552	10	882959.2	
2014-01-25	0	11.513846158147691	15.034825282381665	7.98489510689998	45.718084942339374	2.326241138672448	28.58374983872281	2014	1	1	7.049938095481677	10	718079.9	
2014-01-26	0	11.041772157310871	15.6958736831187	6.38410681937481	43.88329489291598	1.17333333863933	25.987883196648015	2014	1	1	9.318978486373588	10	646878.5	
2014-01-27	0	8.971794807839898	12.36923878339324	5.71509973222898	54.465	2.956132763219946	15.76166655548466	2014	1	0	6.797720818170272	10	773487.44	
2014-01-28	0	6.920916917341572	10.214756447488282	3.622863832012529	56.236577718128885	3.165582181414954	4.947883353864954	2014	1	0	6.592693415467753	10	811722.1	
2014-01-29	0	6.60841654111411	9.88359486870487	3.328958627724018	45.63513513513514	4.072727277520157	28.719166954358418	2014	1	0	6.554636232980852	10	889996.1	
2014-01-30	0	6.88862678787783	9.691825642577314	2.4683768727763686	43.73480673480673	3.336191875925871	28.148833616256714	2014	1	0	7.222449568888945	10	818118.6	
2014-01-31	0	6.4186633838189	12.03897582161886	4.771558498338491	37.01883581683582	1.9878268898218954	39.41788294585776	2014	1	0	7.26742532338937	10	883838.8	

Y comprobado el resultado mediante los datos,

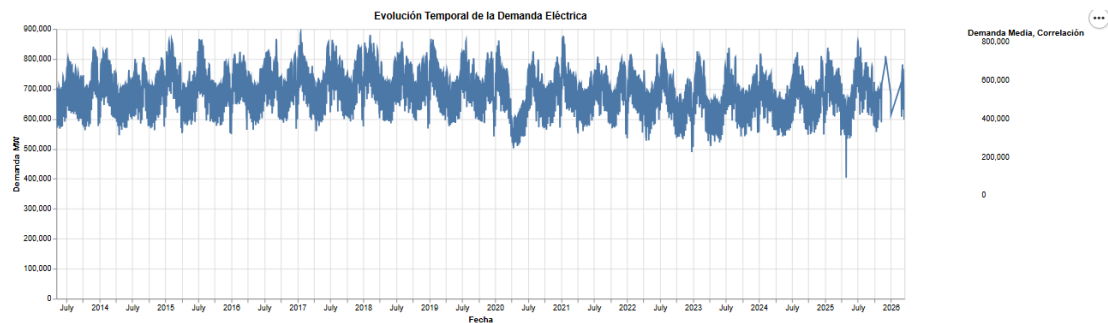
Guardamos los datos ya limpios y con el dataset completo a la capa Oro

/ gold / ENERGIA_GOLD /

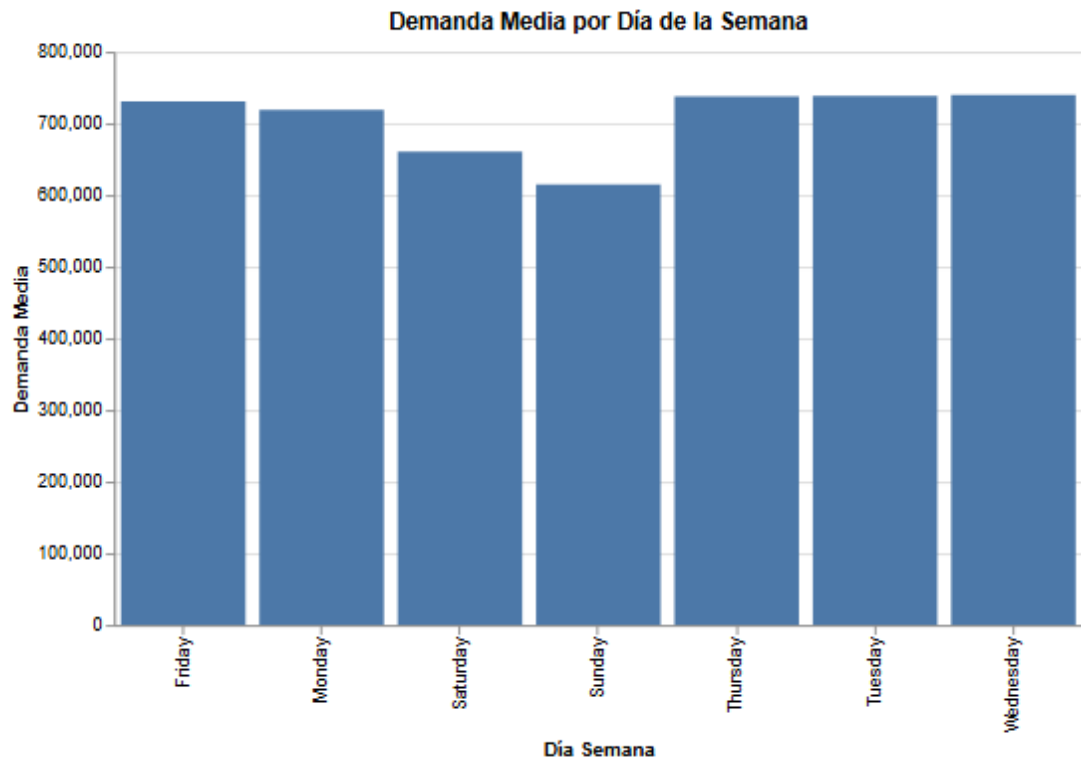
Name	Modified
 _SUCCESS	20h ago
 part-00000-1e397f...	20h ago

Realizamos un dashboard interactivo que nos permita sacar conclusiones mediante la visualización de los datos.

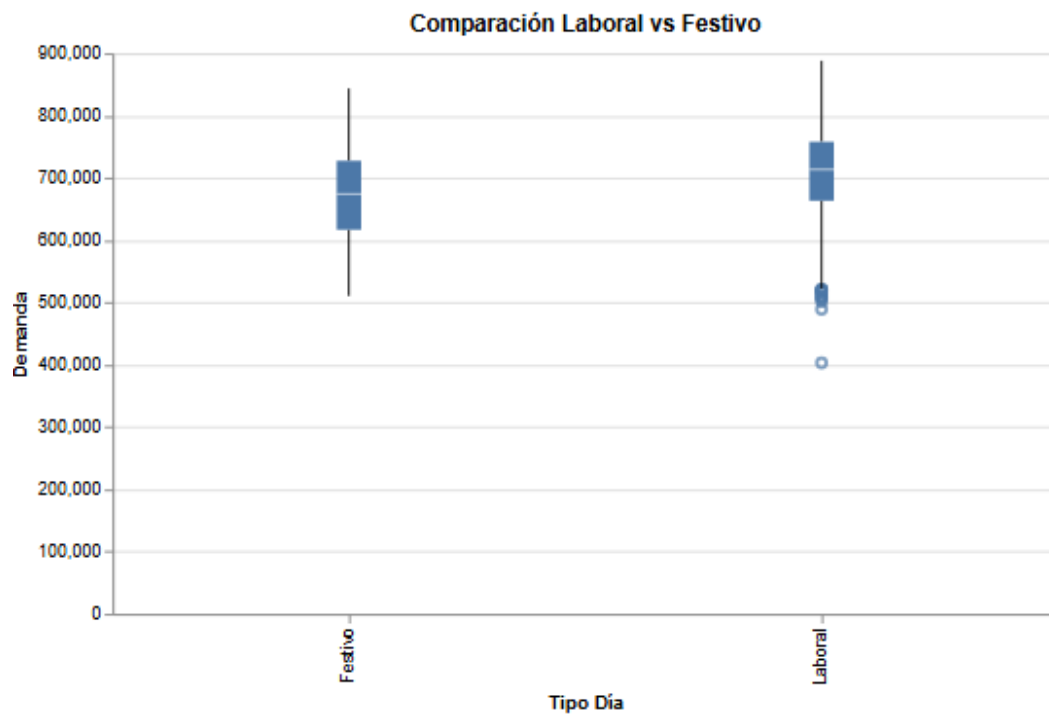
En este dashboard, lo primero que vemos es una evolución temporal de la demanda eléctrica, hemos elegido graficar esto para detectar visualmente picos de consumo, tendencias o caídas de demanda de forma clara.



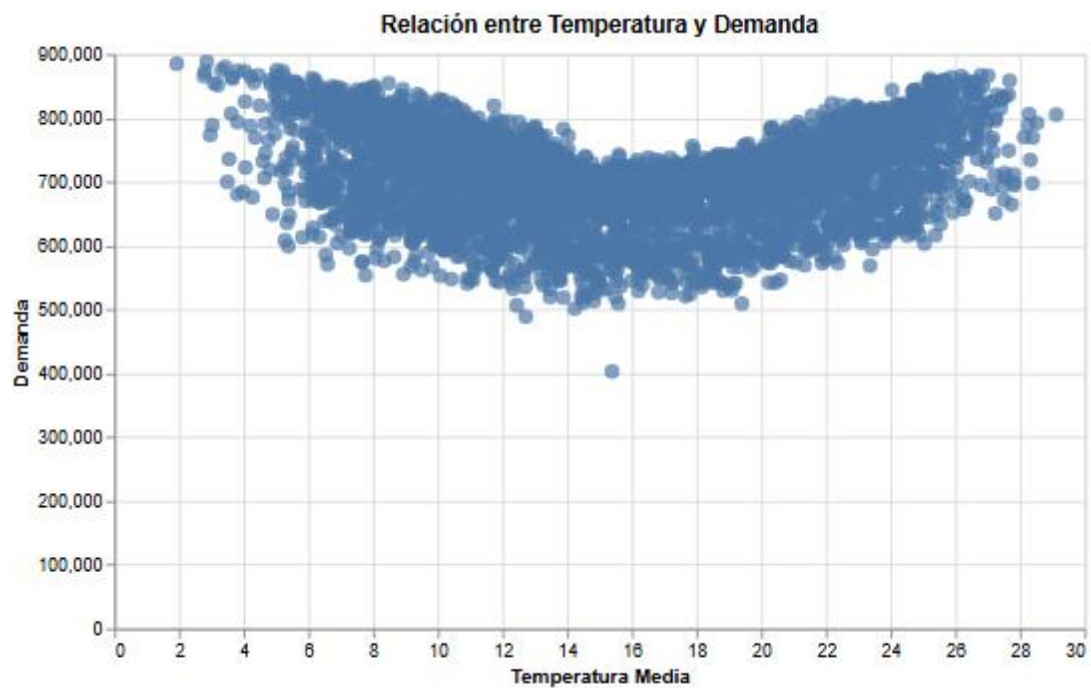
El siguiente gráfico nos permiten ver la demanda media por día de la semana, elegimos este tipo de grafica para observar que días presentan mayor o menos consumo eléctrico



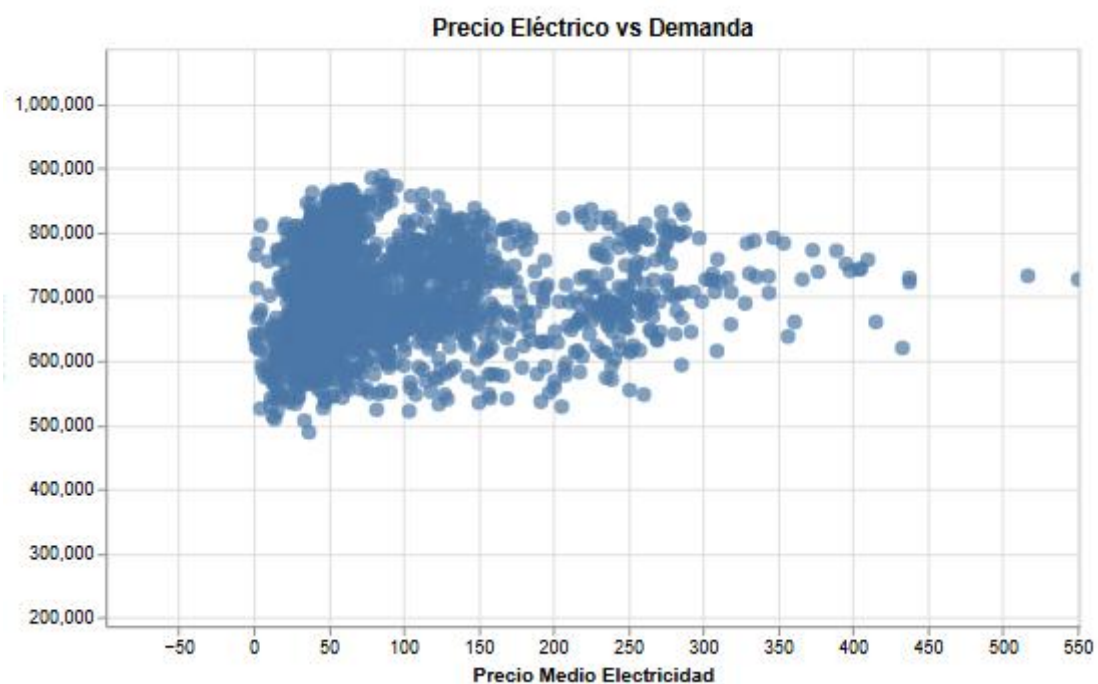
El tercer gráfico que tenemos es un boxplot que nos permite visualizar valores extremos, siendo útil para comparar comportamientos estadísticos entre dos grupos diferentes



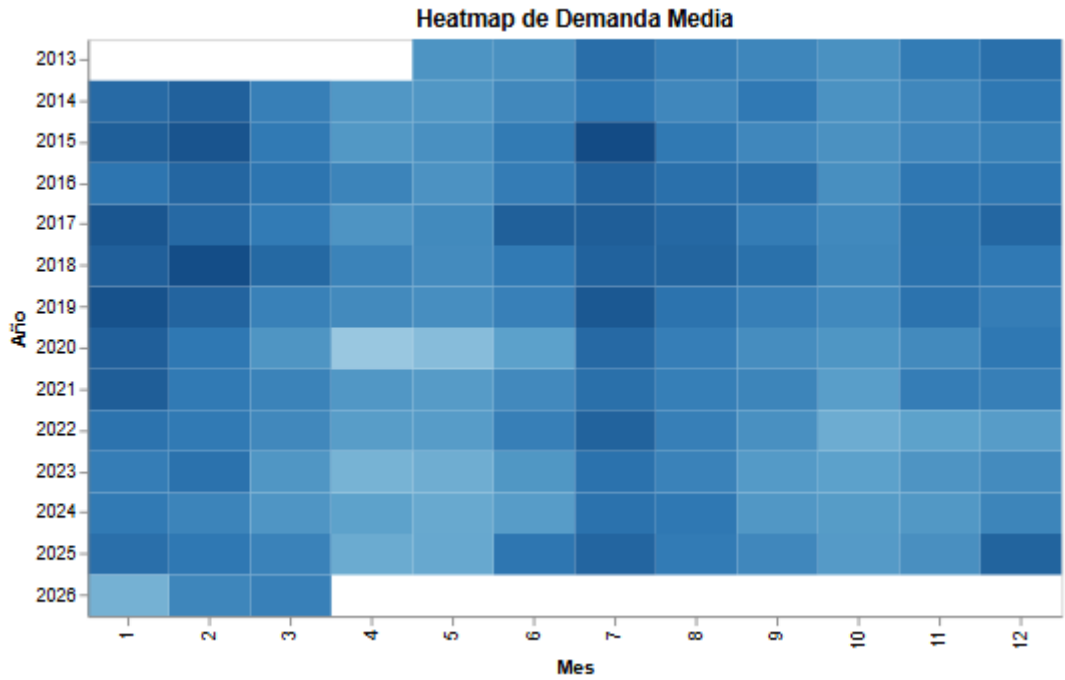
Continuamos con un gráfico de dispersión que relaciona la temperatura y la demanda, este gráfico nos permite representar con puntos y captar la relación entre (ver si existe correlación) entre las dos variables



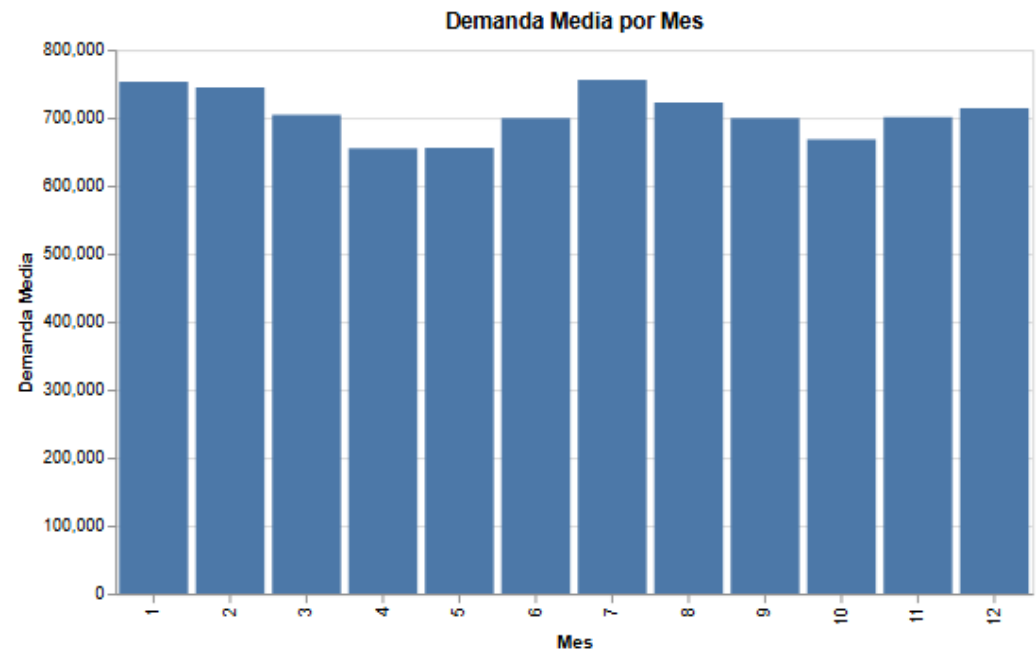
El siguiente gráfico es otro gráfico de dispersión, pero esta vez con las variables de precio y demanda



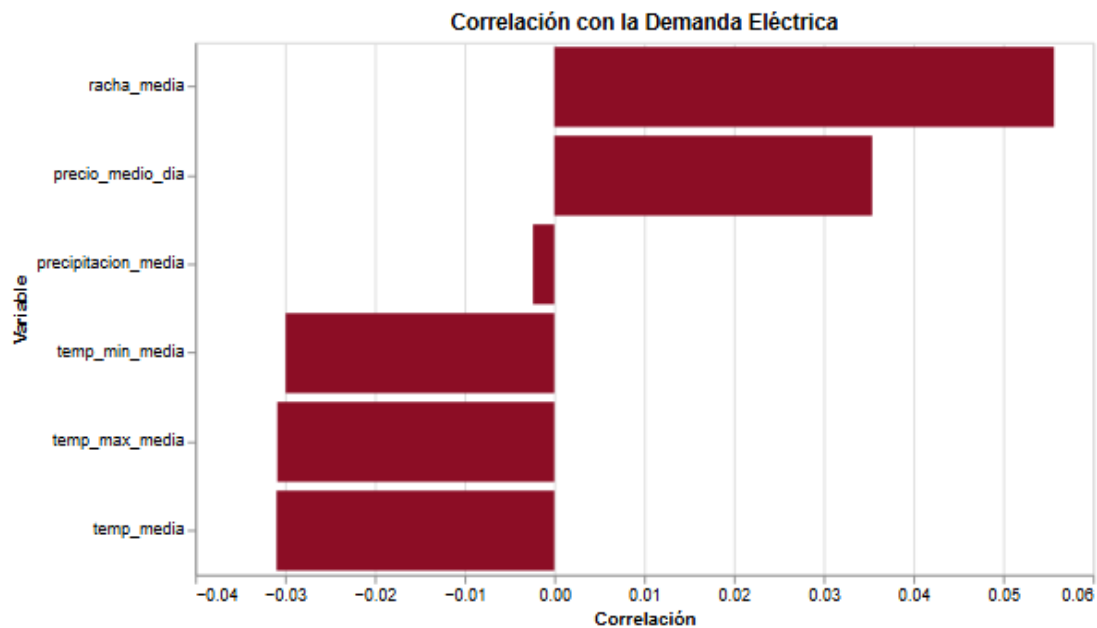
El siguiente gráfico es un mapa de calor de la demanda media, este tipo de gráfico nos permite detectar patrones estacionales y cambios de demanda entre distintos periodos temporales.



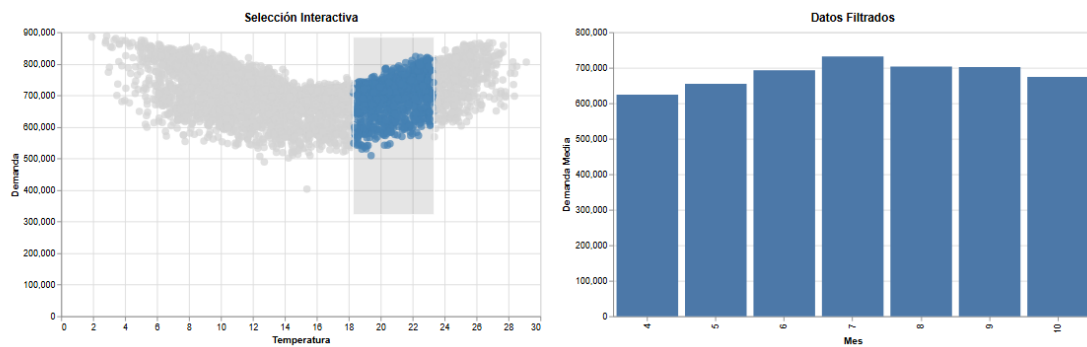
Además, también contamos con un gráfico de barras con la demanda media por mes



Otro gráfico que generamos es uno de correlación con la demanda eléctrica en formato de barras horizontales con el resto de variables



Por último, contamos con dos gráficos, el primero es un gráfico de dispersión entre temperatura y demanda en el que puedes seleccionar un conjunto de datos para visualizar en el siguiente gráfico que es uno de barras



El flujo completo se realiza mediante una función (a excepción de la recogida de datos que está vinculada con un EventBridge)

☐	mensualRecogidaDatos		Estándar programado	default		arn:aws:events:us-east-1:891377024918:rule/mensualRecogidaDatos	-
☐	recogidaFestivosAnual		Estándar programado	default		arn:aws:events:us-east-1:891377024918:rule/recogidaFestivosAnual	-

recogidaFestivosAnual

[Editar](#)[Desactivar](#)[Eliminar](#)[Plantilla de CloudFormation](#)

Detalles de la regla [Información](#)

Nombre de la regla
recogidaFestivosAnual

Descripción
-

Estado

ARN de la regla
 arn:aws:events:us-east-1:891377024918:rule/recogidaFestivosAnual

Nombre del bus de eventos
default

ARN del bus de eventos
 arn:aws:events:us-east-1:891377024918:event-bus/default

Tipo
Estándar programado

Programación de eventos [Información](#)

[Destinos](#)[Supervisión](#)[Etiquetas](#)[Editar](#)

Expresión Cron

00 11 1 ? *

Fechas de los siguientes 10 desencadenadores

UTC

Fri, 01 Jan 2027 01:00:00 UTC
Sat, 01 Jan 2028 01:00:00 UTC
Mon, 01 Jan 2029 01:00:00 UTC
Tue, 01 Jan 2030 01:00:00 UTC
Wed, 01 Jan 2031 01:00:00 UTC
Thu, 01 Jan 2032 01:00:00 UTC
Sat, 01 Jan 2033 01:00:00 UTC
Sun, 01 Jan 2034 01:00:00 UTC
Mon, 01 Jan 2035 01:00:00 UTC
Tue, 01 Jan 2036 01:00:00 UTC

recogidaFestivosAnual

[Editar](#)[Desactivar](#)[Eliminar](#)[Plantilla de CloudFormation](#)

Detalles de la regla [Información](#)

Nombre de la regla
recogidaFestivosAnual

Descripción
-

Estado

ARN de la regla
 arn:aws:events:us-east-1:891377024918:rule/recogidaFestivosAnual

Nombre del bus de eventos
default

ARN del bus de eventos
 arn:aws:events:us-east-1:891377024918:event-bus/default

Tipo
Estándar programado

Programación de eventos [Información](#)

[Destinos](#)[Supervisión](#)[Etiquetas](#)[Editar](#)

Expresión Cron

00 11 1 ? *

Fechas de los siguientes 10 desencadenadores

UTC

Fri, 01 Jan 2027 01:00:00 UTC
Sat, 01 Jan 2028 01:00:00 UTC
Mon, 01 Jan 2029 01:00:00 UTC
Tue, 01 Jan 2030 01:00:00 UTC
Wed, 01 Jan 2031 01:00:00 UTC
Thu, 01 Jan 2032 01:00:00 UTC
Sat, 01 Jan 2033 01:00:00 UTC
Sun, 01 Jan 2034 01:00:00 UTC
Mon, 01 Jan 2035 01:00:00 UTC
Tue, 01 Jan 2036 01:00:00 UTC

```
def CapaOro():
    demandaPlata()
    time.sleep(10)
    preciosPlata()
    time.sleep(10)
    festivosPlata()
    time.sleep(10)
    climaPlata()
    print("Todos los datos han sido recogidos y tratados de manera individual")
    time.sleep(10)
    unionDatos()
    print("Muestreo de datos")
    muestreoDatos()
    graficos = generar_dashboard()
    graficos
    print("Datos unidos en la capa oro \nProceso finalizado")
```